

Лекция № 13

Планирование процессов

Планирование процессов — это один из ключевых аспектов работы операционных систем, который отвечает за распределение процессорного времени между множеством процессов, выполняющихся на компьютере. Главная цель планирования — это эффективное использование процессора, поддержание многозадачности и обеспечение справедливости между процессами.

Планирование процессов позволяет операционной системе управлять процессами таким образом, чтобы они выполнялись в оптимальном порядке и с соблюдением заданных критериев, таких как приоритеты, время выполнения и потребности в ресурсах.

В этой лекции мы рассмотрим, что такое планирование процессов, основные алгоритмы планирования, виды планировщиков и их характеристики.

Основы планирования процессов

- Процесс — это программа, которая выполняется в операционной системе. Каждый процесс может иметь различные состояния:
- Новый — процесс только что создан.
- Готовый — процесс готов к выполнению, но ожидает своей очереди для использования процессора.
- Выполняющийся — процесс активно использует процессор.
- Ожидающий — процесс ожидает завершения какого-то события (например, ввода/вывода).
- Завершенный — процесс завершил свою работу.

Планировщик — это компонент операционной системы, который решает, какой процесс будет выполняться следующим. Планировщик управляет процессами, переводя их из одного состояния в другое, основываясь на политике планирования и алгоритмах.

Основные цели планирования процессов

Планирование процессов имеет несколько целей:

- Эффективность использования процессора — максимизация времени работы процессора без простоя.
- Отзывчивость системы — минимизация времени отклика на запросы пользователя.
- Справедливость — равномерное распределение процессорного времени между процессами.
- Приоритетность — выполнение более важных или срочных процессов в первую очередь.
- Многозадачность — одновременное выполнение нескольких процессов (или имитация этого в однозадачных системах).

Алгоритмы планирования процессов

Существует множество алгоритмов планирования, каждый из которых подходит для разных типов задач и условий эксплуатации системы. Рассмотрим основные из них.

Планирование с использованием очереди FIFO (First In, First Out)

Алгоритм FIFO — это самый простой способ планирования, при котором процессы выполняются в том порядке, в котором они поступили в очередь готовых процессов. То есть первый процесс, который поступил, будет выполняться первым.

Преимущества:

- Простота реализации.
- Подходит для систем, где процессы имеют примерно одинаковые ресурсы и требования.

Недостатки:

- Могут возникать проблемы с производительностью, если один процесс занимает слишком много времени (эффект «голодания»).
- Не учитывает приоритеты процессов и не оптимизирует время отклика.

Планирование с использованием алгоритма кратчайшего времени до завершения (SJF - Shortest Job First)

В этом алгоритме планировщик выбирает процесс с минимальным временем выполнения. Этот алгоритм идеально подходит для систем с фиксированным временем выполнения задач.

Преимущества:

- Минимизирует среднее время ожидания.
- Хорошо работает, когда известны времена выполнения процессов.

Недостатки:

- Сложно точно определить время завершения процесса заранее.
- Может привести к проблемам с «голоданием» для процессов с более длительным временем выполнения.

Планирование с приоритетами

В алгоритме с приоритетами каждому процессу присваивается приоритет, и процесс с наивысшим приоритетом выполняется первым. Приоритеты могут быть назначены вручную или вычисляться операционной системой.

Преимущества:

- Возможность выделить более важные задачи.
- Справедливость в том, что более важные процессы получают процессорное время в первую очередь.

Недостатки:

- Проблемы с «голоданием» для низкоприоритетных процессов.
- Если система не управляет динамическими приоритетами, то низкоприоритетные процессы могут никогда не выполняться.

Алгоритм времени с квантованием (Round Robin, RR)

Алгоритм Round Robin предполагает, что каждому процессу дается фиксированное количество времени (квант), в течение которого процесс может использовать процессор. После истечения времени квантового процесса, операционная система переключается на следующий процесс в очереди.

Преимущества:

- Обеспечивает справедливость, так как каждый процесс получает одинаковое время для выполнения.
- Хорошо подходит для многозадачных систем с большим количеством процессов.

Недостатки:

- Могут возникать проблемы с производительностью, если процессы требуют разных объемов процессорного времени.
- Кратковременные процессы могут занимать большое количество времени из-за накладных расходов на переключение контекста.

Планирование с использованием мультиуровневых очередей (Multilevel Queue Scheduling)

В этом алгоритме процессы разделяются на несколько категорий, и для каждой категории используется свой алгоритм планирования. Например, можно выделить два уровня: один для интерактивных процессов, а другой — для вычислительных.

Преимущества:

- Система может быть более гибкой, оптимизируя выполнение различных типов процессов.

- Уменьшается время отклика для высокоприоритетных процессов.

Недостатки:

- Требуется дополнительных ресурсов для организации очередей и механизма переключения между ними.

- Сложность в управлении и настройке.

Проблемы планирования процессов

Несмотря на эффективность алгоритмов планирования, существуют несколько важных проблем, с которыми сталкиваются операционные системы:

Проблема «голодания» (Starvation)

«Голодание» — это ситуация, когда процесс не получает процессорного времени из-за того, что другие процессы постоянно имеют более высокий приоритет. Особенно это актуально для алгоритмов с приоритетами, где низкоприоритетные процессы могут быть заблокированы.

Проблема инверсии приоритетов (Priority Inversion)

Инверсия приоритетов происходит, когда процесс с низким приоритетом блокирует процесс с высоким приоритетом, захватывая ресурсы, которые требуются высокоприоритетному процессу. Это может вызвать серьезные задержки в выполнении важных задач.

Переключение контекста (Context Switching)

Переключение контекста — это процесс, при котором операционная система сохраняет состояние текущего процесса и загружает состояние следующего процесса. Частое переключение контекста может привести к значительным накладным расходам и снижению производительности.

Ожидание ввода-вывода (I/O Bound Waiting)

Процессы, которые сильно зависят от ввода/вывода, могут находиться в ожидании ресурсов и занимать процессорное время в пустую. Алгоритмы планирования должны эффективно обрабатывать такие процессы.

Планирование процессов — это неотъемлемая часть многозадачных операционных систем. Правильный выбор алгоритма планирования позволяет повысить производительность, улучшить отклик системы и обеспечить справедливое распределение ресурсов. Для успешного планирования важно учитывать как характеристики системы, так и требования процессов, а также возможные проблемы, такие как «голодание» или инверсия приоритетов.

Знание принципов планирования процессов необходимо для разработки эффективных операционных систем и приложений, которые смогут корректно и быстро выполнять задачи при многозадачности.